GUJARAT UNIVERSITY

PROJECT REPORT

EMPLOYEE LEAVE MANAGEMENT SYSTEM

A web-based application for employee leave applications and administrative approvals

Submitted in partial fulfilment of the requirements for

Bachelor of Computer Applications (BCA)

Course: Full Stack Web Development

Course Code: DSC-M-BCA-471P

Submitted by:

Parmar Bhavik Maheshkumar
Roll No : 4003
&
Solanki Dhruv Manojkumar
Roll No : 4004

Under the guidance of
Naina Asnani

Asia Pacific BCA College
Gujarat University
Academic Year : [2026 - 2027]

DECLARATION

We, **"Parmar Bhavik"," Solanki Dhruv"** hereby declare that the project entitled **“Employee Leave Management System”** has been prepared by us as part of our BCA Full Stack Web Development coursework. This report describes the application structure, features, technologies, database design, and testing of the project.

We submit this project for academic evaluation. The references section acknowledges the external resources used for learning and implementation.

Signatures:

Parmar Bhavik: _____

Solanki Dhruv: _____

Date: _____

Place: _____

COLLEGE CERTIFICATE

This is to certify that the project report entitled “Employee Leave Management System” is submitted by the student named below as part of the academic requirements for the Bachelor of Computer Applications (BCA) programme.

The project demonstrates the use of ASP.NET Core MVC, C#, Razor Views, Entity Framework Core, SQL Server, ASP.NET Core Identity, and REST API concepts.

Student Name: _____

Enrollment No.: _____

Project Guide: _____

Signature: _____

Head of Department: _____

College Seal: _____

Date: _____

Note: Replace this sample certificate page with the official certificate format and signatures provided by your college.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my college and department for providing the opportunity to develop the Employee Leave Management System as part of the BCA Full Stack Web Development course.

I am thankful to my project guide and faculty members for their guidance, feedback, and support throughout the planning and development process. Their suggestions helped me understand the practical use of web development concepts such as MVC architecture, database connectivity, authentication, authorization, and form validation.

I also acknowledge the documentation and learning resources provided by Microsoft and the .NET community, which were useful for understanding ASP.NET Core, Entity Framework Core, and ASP.NET Core Identity.

Finally, I thank everyone who supported me during the preparation of this project and report.

— Student

INDEX

No.	Description	Page No.
1	College Certificate	3
2	Acknowledgement	4
3	Project Overview	6
4	Purpose and Scope of the Project	7
5	Existing vs Proposed System	8
6	Feasibility Study	9
7	Hardware and Software Requirement Specification	10
8	Project Folder Structure	11
9	Core Modules	12
10	Screenshots	13
11	Limitations	17
12	Future Enhancements	18
13	References	19

Page numbers are indicative. Update the index after making final edits or adding the official college certificate and application screenshots.

1. PROJECT OVERVIEW

1.1 Introduction

The Employee Leave Management System (ELMS) is a database-driven web application that organizes employee leave requests and administrative decisions in one place. Employees can sign in, view their dashboard, submit leave applications, track request status, review leave history, and change their password. Administrators can manage employee profiles, inspect leave requests, and approve or reject pending applications.

1.2 Problem Statement

When leave requests are handled through paper forms, messages, or spreadsheets, records can be misplaced, status updates may be delayed, and it can be difficult to identify overlapping leave dates. The proposed system centralizes employee and leave information in a relational database and provides role-based access to the main operations.

1.3 Project Objectives

- Provide a central place for employees to submit and track leave requests.
- Allow administrators to manage employee profiles and review leave applications.
- Store employee details and leave records in SQL Server through Entity Framework Core.
- Use secure sign-in and role-based authorization for Admin and Employee users.
- Validate leave dates and prevent overlapping approved leave periods.
- Demonstrate MVC architecture, Razor Views, database migrations, and REST API endpoints.

1.4 Project Summary

Item	Description
Project name	Employee Leave Management System
Academic course	BCA – Full Stack Web Development
Course code	DSC-M-BCA-471P
Application type	Web application
Primary users	Administrator and Employee
Main technology	ASP.NET Core MVC with C#
Database	Microsoft SQL Server / LocalDB
Data access	Entity Framework Core
Authentication	ASP.NET Core Identity
UI approach	Razor Views with plain CSS

2. PURPOSE AND SCOPE OF THE PROJECT

2.1 Purpose

The purpose of ELMS is to make leave management more organized, traceable, and convenient. It reduces the need to maintain separate paper forms or spreadsheets and helps employees and administrators view the current status of each request.

2.2 Project Scope

The current implementation focuses on the core leave workflow and employee record management. It is intended as an academic project and a foundation that can be extended for real organizational use after additional security, deployment, and policy requirements are addressed.

2.3 Included Features

- Login and logout using ASP.NET Core Identity.
- Role-based access for Admin and Employee accounts.
- Employee creation, listing, searching, editing, and details pages.
- Employee login account creation when an administrator creates an employee profile.
- Employee self-service password change requiring the current password.
- Leave application with leave type, start date, end date, and reason.
- Leave status tracking: Pending, Approved, Rejected, and Cancelled.
- Administrator review with an optional comment.
- Validation for past start dates, invalid date ranges, and overlapping approved leave.
- REST API endpoints for employee and leave data.

2.4 Out of Scope

- Payroll processing and salary calculations.
- Attendance, biometric devices, and shift scheduling.
- Automated email/SMS notifications.
- Organization-specific holiday calendars and leave-balance accrual.
- Advanced analytics, audit trails, and multi-branch policy configuration.

2.5 Intended Users

User	Main responsibilities
Administrator	Manage employee profiles; view leave requests; approve or reject pending requests; access protected administrative functions.
Employee	View own dashboard; apply for leave; check own request history and status; cancel pending requests; change own password.

3. EXISTING VS PROPOSED SYSTEM

3.1 Existing System

In a manual or semi-manual process, employees may submit leave requests using paper forms, email, messaging applications, or spreadsheets. The responsible person then records the request and communicates the decision separately.

3.2 Issues in the Existing System

- Requests and approvals may be spread across different places.
- Employees may need to ask an administrator for status updates.
- Duplicate or incomplete records can occur during manual entry.
- Searching old requests can take time.
- Overlapping leave dates may be harder to notice.
- Access to sensitive employee information may not be consistently controlled.

3.3 Proposed System

The proposed ELMS provides a web interface connected to a central SQL Server database. Identity manages user accounts and roles. MVC controllers process requests, validate inputs, and call Entity Framework Core to read or save data. Razor Views present the pages to users, while protected API controllers expose selected data as JSON.

3.4 Comparison

Basis	Existing / Manual System	Proposed ELMS
Record keeping	Paper, messages, or spreadsheets	Central SQL Server database
Request submission	Manual communication	Online form
Status visibility	Requires follow-up	Available through leave history
Approval process	Manual update or message	Admin review and status update
Validation	Depends on manual checking	Application-level validation
Access control	May be informal	Identity authentication and role authorization
Search	Manual scanning	Search and database queries
Extensibility	Difficult to standardize	Can add modules and API endpoints

3.5 Expected Benefits

- Improved visibility of employee leave requests.
- More consistent storage of employee and leave information.
- Clear separation between administrative and employee actions.
- Reduced manual effort for common leave-management tasks.
- A foundation for future HR-related features.

4. FEASIBILITY STUDY

4.1 Technical Feasibility

The project uses the .NET ecosystem, ASP.NET Core MVC, Entity Framework Core, and SQL Server. These technologies support web applications, database access, authentication, and API development. The project can be developed in VS Code with the .NET SDK and a local SQL Server or LocalDB instance.

4.2 Economic Feasibility

The project can be developed using freely available development tools such as the .NET SDK and Visual Studio Code. SQL Server LocalDB can be used for local academic testing. Hosting, production database services, domain registration, and operational support may introduce costs if the system is deployed for real users.

4.3 Operational Feasibility

The system uses familiar web forms, tables, buttons, and status labels. Employees can submit and review their own requests, while administrators manage employee records and make decisions. Basic training and clear usage instructions should be sufficient for a demonstration.

4.4 Schedule Feasibility

The project is modular and suitable for academic development in stages: setup, database design, authentication, employee management, leave workflow, API implementation, testing, and documentation. The final schedule depends on the student's available time and review requirements.

4.5 Legal and Security Considerations

A real deployment should follow applicable privacy and employment rules, use HTTPS, protect database credentials, restrict access to employee information, and apply an organizational leave policy. The academic version should use test data and should not store real sensitive information without approval.

4.6 Feasibility Conclusion

The project is technically and operationally feasible as a BCA web development project. Production use would require further testing, deployment planning, policy configuration, monitoring, and a formal security review.

5. HARDWARE AND SOFTWARE REQUIREMENT SPECIFICATION

5.1 Hardware Requirements

Component	Minimum practical requirement
Processor	Dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 2 GB free for tools and project files, plus database space
Display	1366 × 768 resolution or higher recommended
Network	Internet connection for initial tool/package installation; local network optional

5.2 Software Requirements

Software	Purpose
Windows 10/11 or compatible OS	Development and local execution environment
.NET 10 SDK	Build and run the ASP.NET Core application
Visual Studio Code	Code editor
C# Dev Kit / C# extension	C# language support and debugging
SQL Server LocalDB or SQL Server Express	Relational database
Entity Framework Core tools	Apply migrations and manage schema
Web browser	Access and test the application
Git (optional)	Version control

5.3 Frameworks and Libraries

- ASP.NET Core MVC: routes requests to controllers and views.
- Razor Pages: supports Identity account pages and the password-change page.
- ASP.NET Core Identity: handles login, password hashing, and role management.
- Entity Framework Core: maps C# entities to database tables and performs queries.
- SQL Server provider: connects EF Core to SQL Server.
- Swashbuckle / Swagger: documents and displays available API endpoints in Development.

5.4 Configuration Notes

The project targets .NET 10. Its connection string is configured in appsettings.json. For a fresh database, apply the included InitialCreate migration with the Entity Framework command `dotnet ef database update`. Development admin credentials should be provided through environment variables or secure configuration and should never be committed to source control.

6. PROJECT FOLDER STRUCTURE

The main application folder is Employee_Leave_Management_System. The following list summarizes the important files and directories in the supplied project archive.

Folder / File	Purpose
Program.cs	Registers services, configures middleware, routes, Identity, Swagger, and database initialization.
appsettings.json	Contains application configuration, including the database connection string.
EmployeeLeaveManagementSystem.csproj	Defines the target framework and NuGet package references.
Controllers/	MVC controllers for dashboard, employee management, and leave requests.
Controllers/Api/	Protected REST API controllers for employee and leave data.
Models/	Entity classes for ApplicationUser, Employee, and LeaveRequest.
ViewModels/	Form-specific models such as leave application and decision input.
Data/ApplicationDbContext.cs	Entity Framework Core database context.
Data/SeedData.cs	Creates required roles and configured development admin account.
Migrations/	EF Core migration and model snapshot used to create/update database schema.
Areas/Identity/Pages/Account/	Identity account pages, including ChangePassword.
Views/Dashboard/	Admin and employee dashboard views.
Views/Employees/	Employee list, create, edit, details, and delete views.
Views/LeaveRequests/	Leave application, history, and review views.
Views/Shared/	Shared layout and common navigation.
wwwroot/css/site.css	Styles for page layout, forms, tables, and status messages.

6.1 Application Flow

1. The user opens the application in a web browser.
2. ASP.NET Core routing selects the relevant MVC controller or Razor Page.
3. Authentication identifies the signed-in user and authorization checks their role.
4. The controller validates the request and uses ApplicationDbContext or Identity services.
5. Entity Framework Core communicates with SQL Server.
6. The application returns a Razor View or JSON response.

7. CORE MODULES

7.1 Authentication and Authorization

ASP.NET Core Identity manages login, logout, password hashing, and user roles. The application defines Admin and Employee roles. Controller and API attributes restrict protected operations. Password changes are performed through Identity and require the current password.

7.2 Employee Management

The Employees module allows an administrator to view and search employee records, create a profile, edit details, view employee information, and delete a record when it has no leave history. Employee details include full name, email, department, designation, joining date, and the optional linked Identity user ID.

7.3 Employee Dashboard

The employee dashboard is intended to show employee-facing information. If an account is not linked to an employee profile, the application displays a profile-missing message and asks the user to contact the administrator.

7.4 Leave Application

An authenticated employee can apply for Casual, Sick, or Earned leave. The form collects leave type, start date, end date, and reason. The application checks that the start date is not in the past, the end date is not before the start date, and the request does not overlap an already approved leave period.

7.5 Leave History and Status Tracking

Leave requests are stored with the employee ID, leave type, date range, reason, application timestamp, status, and optional administrator comment. Status values are Pending, Approved, Rejected, and Cancelled. Employees see their own requests; administrators can review requests across employees.

7.6 Leave Review and Decision

Administrators can open a pending request and approve or reject it, optionally adding a comment. Only pending requests can be reviewed. Before approval, the system checks for overlap with another approved request for the same employee.

7.7 REST API

Method	Route	Access / purpose
GET	/api/employees	Admin: list employee records
GET	/api/employees/{id}	Admin: retrieve one employee record
GET	/api/leaves	Authenticated: list leave records; employees are limited to their own
GET	/api/leaves/{id}	Authenticated: retrieve a leave request subject to ownership checks
POST	/api/leaves	Employee/Admin: submit a leave request using JSON

POST	/api/leaves/{id}/decision	Admin: approve or reject a pending request

The Development environment enables Swagger UI at `/swagger`. Protected API endpoints require authentication. The API should be tested using an appropriate authenticated session or a configured API authentication flow.

7.8 Database Design

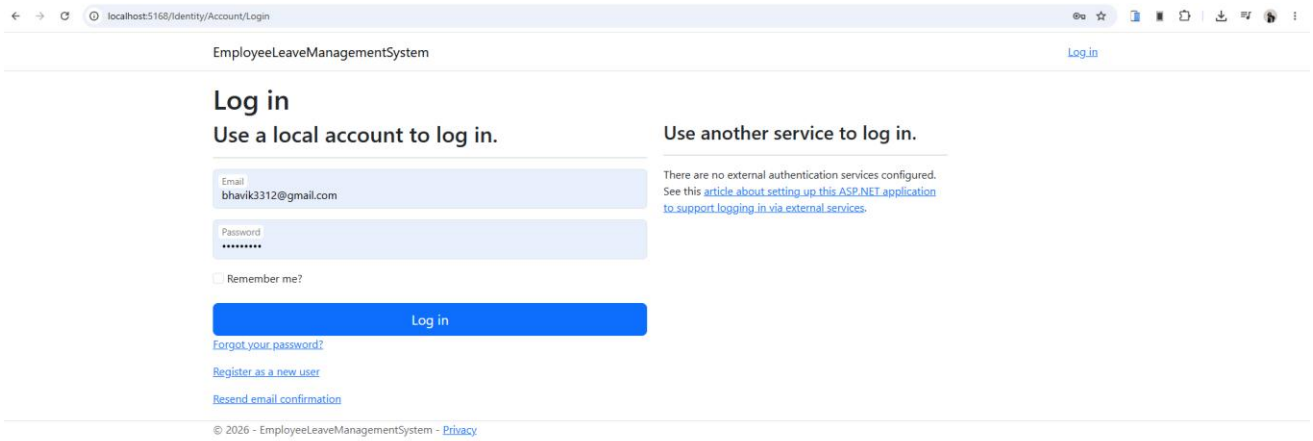
Entity / table	Key information
Identity tables	User accounts, password hashes, roles, and role assignments.
Employees	Employee profile and optional one-to-one link to an Identity user.
LeaveRequests	Leave type, dates, reason, status, timestamp, and admin comment.

One employee can have multiple leave requests. An Identity user can be linked to zero or one employee profile. Employee deletion is blocked when leave history exists.

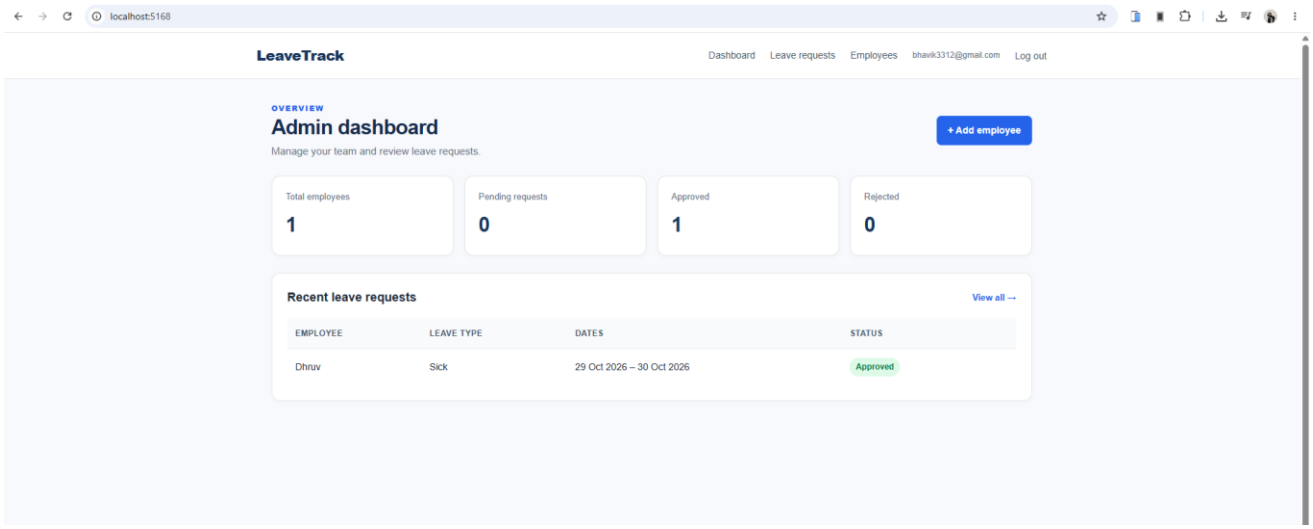
8. SCREENSHOTS

Insert actual screenshots from the running application in the spaces below. Use screenshots captured from your own project; do not submit sample or imagined screens as proof of execution.

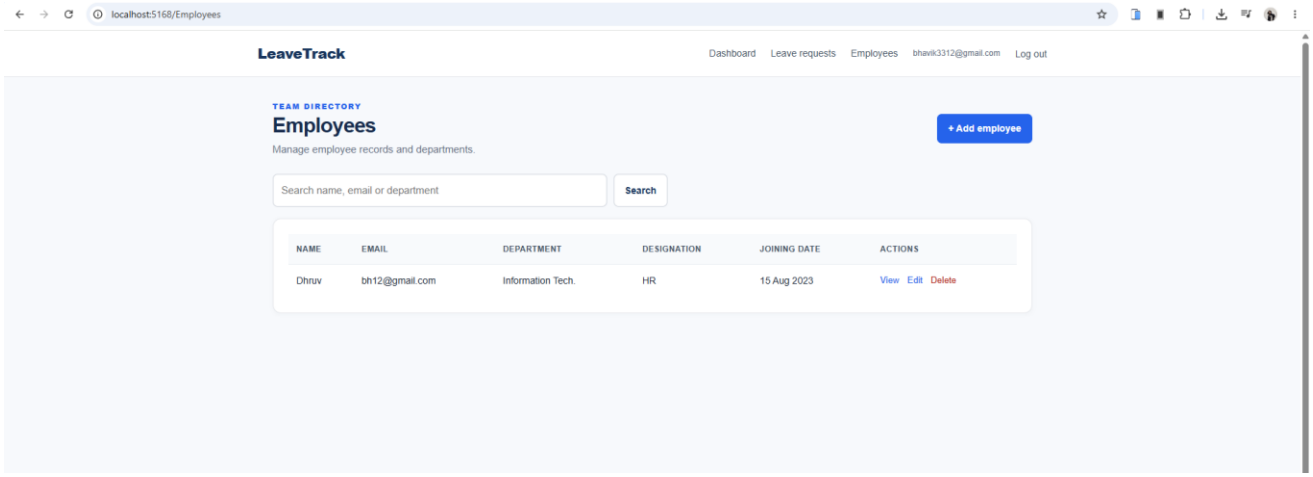
8.1 Login Page



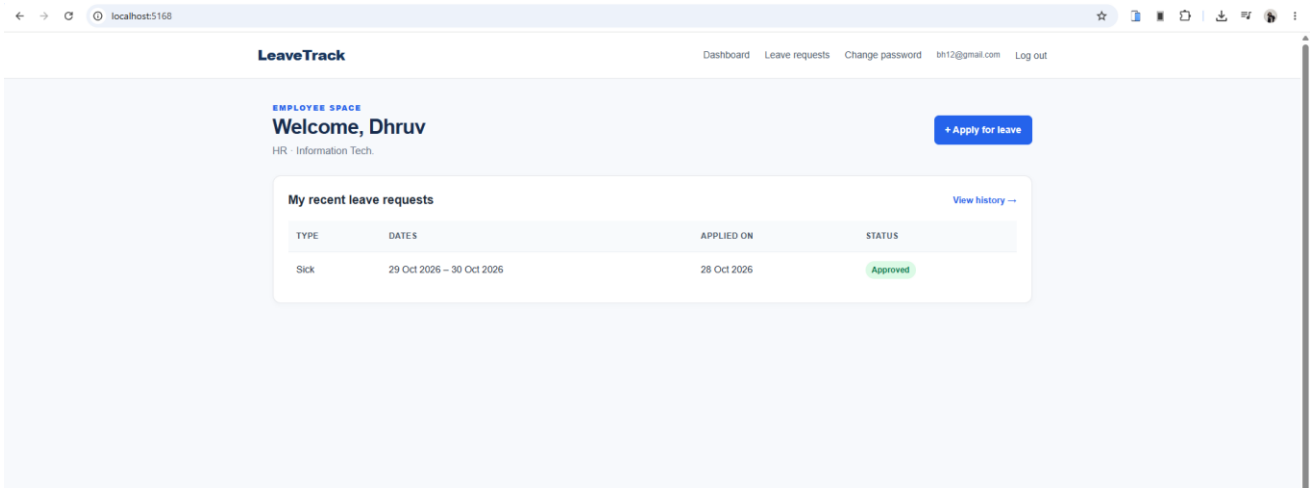
8.2 Admin Dashboard



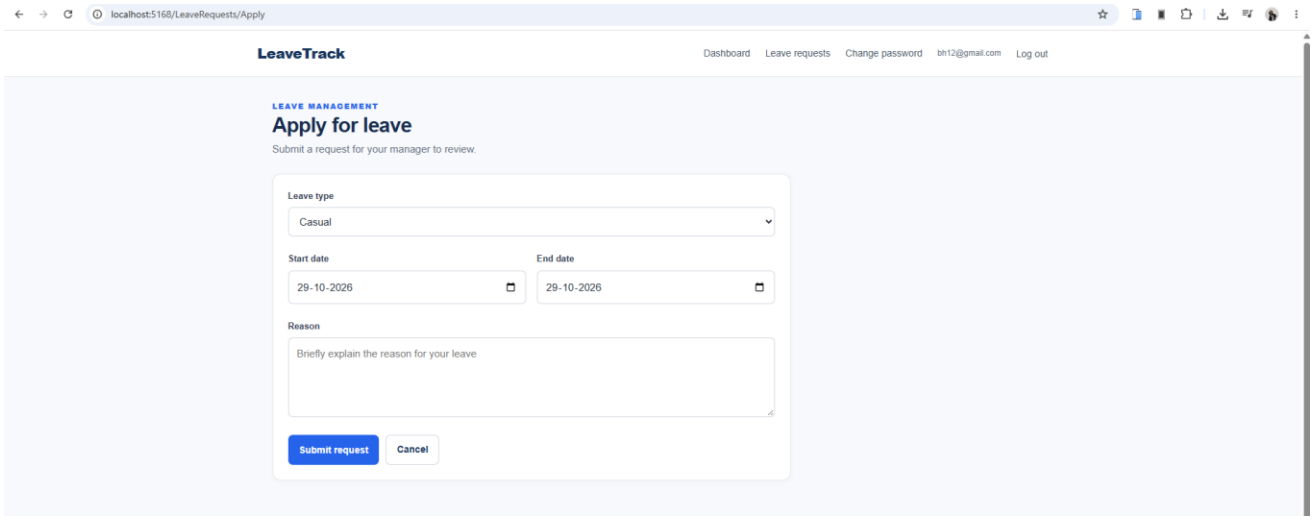
8.3 Employee List / Create Employee



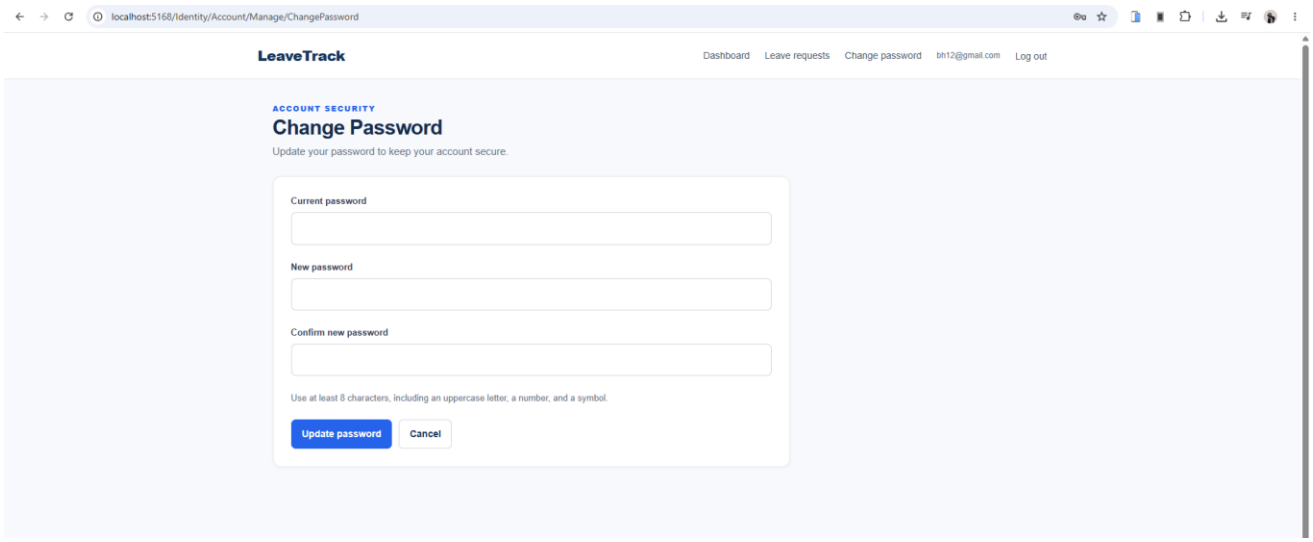
8.4 Employee Dashboard



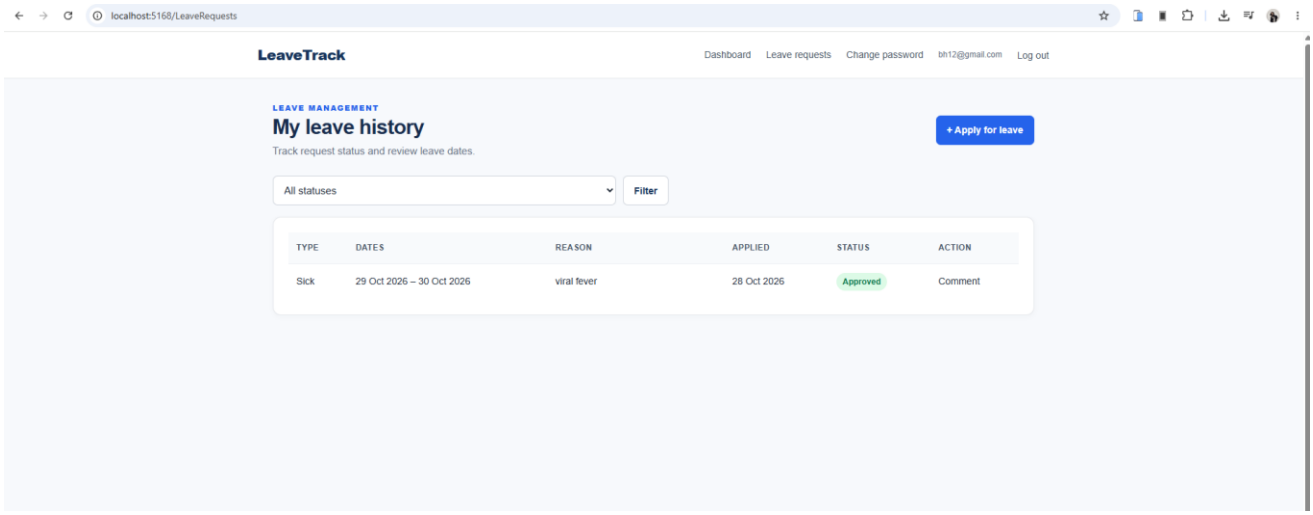
8.5 Apply for Leave



8.6 Leave History



8.8 Change Password



9. LIMITATIONS

- Email and SMS notifications are not implemented in the current version.
- Leave balances, accrual rules, and organization-specific holiday calendars are not included.
- Attendance, payroll, and shift management are outside the project scope.
- Advanced audit logging and detailed reporting are not implemented.
- API routes are protected and require an appropriate authentication setup for external clients.
- The project has not been independently security-audited for production deployment.
- Successful build and database tests must be confirmed on the target machine with the required .NET SDK and SQL Server installed.

9.1 Risk Mitigation

Use test accounts during demonstrations, keep passwords private, configure HTTPS, store secrets outside source code, back up the database, and test both Admin and Employee permissions before presenting the application.

10. FUTURE ENHANCEMENTS

- Email notifications for submission, approval, rejection, and cancellation.
- Leave balance tracking and configurable leave policies.
- Holiday calendar and organization-specific working days.
- Monthly and yearly leave reports with export to PDF or Excel.
- Audit logs for employee changes and administrative decisions.
- Department-level approval workflows and multiple administrator roles.
- Profile editing and account recovery features with secure verification.
- Improved API authentication for external applications.
- Automated unit, integration, and end-to-end tests.
- Deployment to a secure hosting platform with HTTPS, logging, backups, and monitoring.

10.1 Conclusion

The Employee Leave Management System demonstrates a practical use of full-stack web development concepts. It combines an MVC web interface, role-based authentication, relational data storage, leave validation, administrative review, and REST API endpoints. The project provides a useful academic foundation that can be expanded with organization-specific policies, notifications, reports, and production-grade operational controls.

11. REFERENCES

- Microsoft Learn — ASP.NET Core documentation: https://learn.microsoft.com/aspnet/core
- Microsoft Learn — ASP.NET Core Identity: https://learn.microsoft.com/aspnet/core/security/authentication/identity
- Microsoft Learn — Entity Framework Core: https://learn.microsoft.com/ef/core
- Microsoft Learn — SQL Server provider for EF Core: https://learn.microsoft.com/ef/core/providers/sql-server
- Microsoft Learn — ASP.NET Core Web API: https://learn.microsoft.com/aspnet/core/web-api
- Swashbuckle.AspNetCore / Swagger documentation: https://github.com/domaindrivendev/Swashbuckle.AspNetCore